TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA HỆ THỐNG THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

Học phần: CSE703093 — An toàn phần mềm

Kiểm chứng end-to-end đặc tả hệ thống thanh toán điện tử: từ mô hình Kripke tới fuzzing

Hướng đề tài: Chủ đề 5 — Quy trình end-to-end tích hợp cả 4 chương

Sinh viên thực hiện:	**[TODO: Họ và tên]**
Mã số sinh viên:	**[TODO: MSSV]**
Lớp:	**[TODO: Lớp]**
Giảng viên hướng dẫn:	ThS. Vũ Quang Dũng
Học kỳ:	**[TODO: Học kỳ, năm học]**

Hà Nội, **[TODO: tháng/năm]**

Mục lục

1 Giới thiệu và Mục tiêu **1**

1.1 Bối cảnh . . . 1
1.2 Mục tiêu cụ thể . . . 1
1.3 Phạm vi . . . 1
1.4 Cấu trúc báo cáo . . . 2

2 Mô tả hệ thống mục tiêu **3**

2.1 Chức năng và luồng xử lý . . . 3
2.2 Bề mặt tấn công (attack surface) . . . 4
2.3 Mô hình mối đe dọa (STRIDE) . . . 4
2.4 Đặc tả yêu cầu . . . 4

3 Phương pháp thực hiện **6**

3.1 Chương 1 — Mô hình Kripke và kiểm chứng CTL . . . 6
3.2 Chương 3 — Bounded Model Checking bằng Z3 . . . 7
3.3 Chương 2 — Phân tích an toàn bộ nhớ trên mã C . . . 7
3.4 Chương 3 — Trích xuất AST và kiểm chứng SMT . . . 8
3.5 Chương 4 — Fuzzing . . . 8
3.6 Gộp kết quả, ánh xạ CWE và đánh giá test case . . . 9

4 Kết quả và Phân tích **10**

4.1 Kết quả Chương 1 — Kripke và CTL . . . 10
4.1.1 Bounded Model Checking . . . 12
4.2 Phân tích từng lỗ hổng và bản vá . . . 12
4.2.1 **CWE-120** (Buffer Copy without Checking Size of Input) tại `parse_webhook_signature` . . . 12
4.2.2 **CWE-415** (Double Free) tại `close_payment_session` . . . 13
4.2.3 **CWE-416** (Use After Free) và **CWE-401** (Memory Leak) . . . 14
4.2.4 **CWE-190** (Integer Overflow or Wraparound) . . . 14
4.2.5 **CWE-369** (Divide By Zero) tại `refund_ratio` và **CWE-798** (Hard-coded Credentials) . . . 15
4.3 So sánh các chiến lược fuzzing . . . 16
4.4 Bảng tổng hợp trước/sau . . . 17
4.5 Truy vết test case . . . 18

5 Thảo luận và Hạn chế **19**
5.1 Tính đúng đắn (sound) và đầy đủ (complete) của từng kỹ thuật 19
5.2 Hạn chế 19
5.3 Phản biện và bài học 20

6 Kết luận **21**

A Hướng dẫn chạy lại **23**

B Danh mục bằng chứng nộp kèm **24**

C Khai báo sử dụng công cụ AI **25**

Danh sách hình vẽ

2.1 Kiến trúc luồng thanh toán; đường đỏ là dữ liệu vượt trust boundary. 3

3.1 Pipeline kiểm chứng end-to-end từ đặc tả tới báo cáo CWE. 6

4.1 Mô hình Kripke của đặc tả v0 (sinh tự động). Trạng thái đỏ là trạng thái vi phạm; cạnh đỏ là phản ví dụ đầu tiên (**AG** ¬*violation*). 10

Danh sách bảng

2.1 Các điểm nhận dữ liệu từ bên ngoài và rủi ro tương ứng. 4
2.2 Phân tích STRIDE cho luồng thanh toán. 4
2.3 Yêu cầu trong sheet `YeuCauChucNang` (rút gọn). 5
2.4 Quy mô đặc tả (v0 / v1). 5

3.1 Thuộc tính CTL và truy vết tới yêu cầu. 7

4.1 Kết quả kiểm chứng CTL trên đặc tả v0 và v1 (trích `comparison.md`). 11
4.2 Kết quả BMC (trích `logs/bmc.txt`). 12
4.3 Kết quả SMT (trích `logs/z3.txt`); mọi biến trong miền int32. 15
4.4 Fuzzing với ngân sách 2 000 lần thử, seed $= 1$ (trích `logs/fuzz.csv`). 16
4.5 Tổng hợp lỗ hổng và trạng thái trước/sau (rút gọn từ `before_after.md` và `comparison.md`). 17
4.6 Test case truy vết từ đặc tả, đánh giá tự động (trích `comparison.md`). 18

5.1 Tính chất của các kỹ thuật đã dùng trong đề tài. 19

1. Giới thiệu và Mục tiêu

1.1 Bối cảnh

Phần mềm thanh toán là nơi một lỗi logic nhỏ chuyển thành thiệt hại tài chính trực tiếp: giao hàng khi tiền chưa thực sự được thu, ghi nhận thu tiền hai lần khi cổng thanh toán gửi lại webhook, hoặc chấp nhận một webhook giả mạo nhờ một "mã bí mật" bị bỏ quên trong mã nguồn. Các lỗi này thuộc nhóm **CWE-840** (Business Logic Errors), **CWE-798** (Hard-coded Credentials) và **CWE-667** (Improper Locking) đã học ở Chương 1. Bên cạnh đó, các hàm xử lý dữ liệu từ cổng thanh toán (phân tích chữ ký webhook, cộng dồn số tiền) còn chịu các lớp lỗi bộ nhớ và số học của Chương 2–3.

Báo cáo áp dụng tuần tự kỹ thuật của cả bốn chương lên *một* hệ thống — cổng thanh toán đơn hàng thương mại điện tử `payment_gateway` — theo hướng Chủ đề 5 (quy trình end-to-end) của Hướng dẫn Bài tập lớn. Hệ thống được đặc tả hai lần: **v0** là thiết kế ban đầu còn lỗi, **v1** là thiết kế đã sửa theo kết quả kiểm chứng.

1.2 Mục tiêu cụ thể

1. **Ch.1 — Mô hình hóa:** dựng cấu trúc Kripke $K = (S, S_0, R, L)$ cho vòng đời thanh toán và khóa sổ cái; kiểm chứng 8 công thức CTL (an toàn, sống còn, khả đạt) và 5 cặp loại trừ lẫn nhau, có phản ví dụ cho mọi vi phạm.
2. **Ch.3 — BMC:** kiểm chứng lại các thuộc tính an toàn bằng Bounded Model Checking với Z3, xác định completeness threshold để kết luận "an toàn" là đầy đủ.
3. **Ch.2 — Bộ nhớ:** phát hiện và vá 4 lỗi bộ nhớ C (tràn bộ đệm, rò rỉ, double free, use-after-free), xác nhận trước/sau bằng static scanner và AddressSanitizer.
4. **Ch.3 — SMT:** chứng minh khả năng tràn int32 của 6 bộ đếm/tổng tiền, và chứng minh tiền điều kiện của bản vá loại bỏ hoàn toàn tràn số (UNSAT).
5. **Ch.4 — Fuzzing:** so sánh định lượng black-box, coverage-guided và concolic fuzzing với cùng ngân sách 2 000 lần thử trên 3 hàm Python.

1.3 Phạm vi

Trong phạm vi: một đơn hàng, thanh toán toàn phần qua một cổng; hoàn tiền toàn phần; khóa sổ cái giữa hai worker; 9 hàm C/Python sinh ra từ đặc tả (khoảng 340 dòng mã C cho hai phiên bản); kiểm chứng trên mô hình và trên mã sinh ra.

Ngoài phạm vi: hoàn tiền một phần, nhiều cổng thanh toán, tích hợp ngân hàng thật, hiệu năng (các yêu cầu phi chức năng chỉ được tài liệu hóa, không kiểm chứng tự động).

Tuyên bố về lỗi cài đặt có chủ đích

Các lỗi trong đặc tả v0 (cạnh chuyển trạng thái sai trong mô hình, cột `Loi` của sheet `HamSinhMa`) được **cài có chủ đích** để làm đối tượng cho công cụ phát hiện, đúng như Hướng dẫn Bài tập lớn cho phép. Mọi kết luận "phát hiện được" trong báo cáo đều dựa trên *kết quả chạy công cụ*: công cụ gộp CWE chỉ ghi nhận một lỗ hổng khi có bằng chứng (finding của scanner, log sanitizer, model của Z3, input gây lỗi của fuzzer, phản ví dụ CTL/BMC); lỗi đã khai báo mà không công cụ nào tái hiện được sẽ bị đánh dấu *declared-only*. Trong lần chạy cuối, không có lỗi nào thuộc loại này.

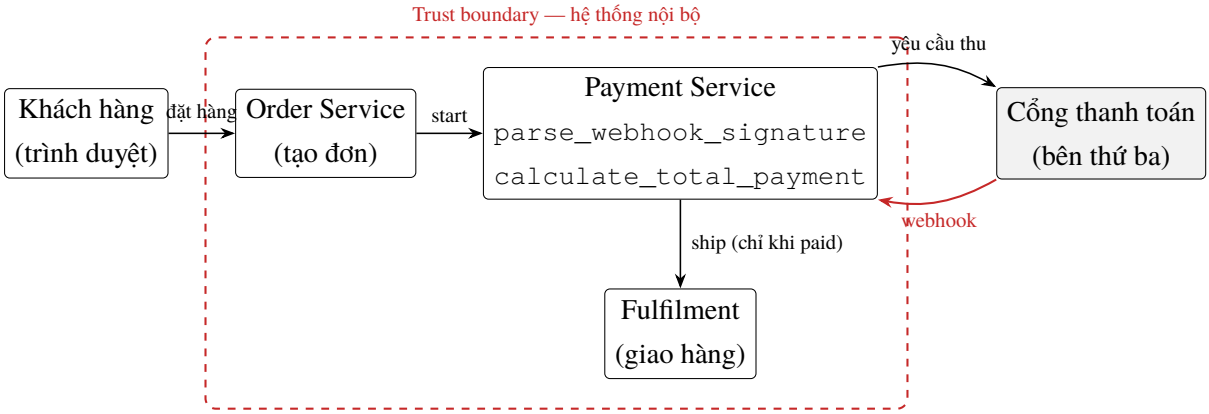
1.4 Cấu trúc báo cáo

Chương 2 mô tả hệ thống mục tiêu; Chương 3 trình bày phương pháp theo từng chương học phần; Chương 4 trình bày kết quả kèm bằng chứng; Chương 5 thảo luận hạn chế; Chương 6 kết luận.

2. Mô tả hệ thống mục tiêu

2.1 Chức năng và luồng xử lý

Đơn hàng được tạo (`CREATED`), chuyển sang chờ cổng thanh toán (`PENDING`). Cổng có thể xác nhận thu tiền (`SUCCEEDED`), báo thất bại (`FAILED`) hoặc không phản hồi (`UNKNOWN` — timeout); đơn timeout phải được đối soát (`RECONCILING`) để biết kết quả thật. Chỉ đơn đã thu tiền mới được giao (`SHIPPED`) hoặc hoàn tiền (`REFUNDING` → `REFUNDED`). Song song, worker xử lý webhook và worker đối soát cùng ghi vào sổ cái, được bảo vệ bởi một khóa. Kết quả từ cổng về qua *webhook* — điểm nhận dữ liệu từ bên ngoài quan trọng nhất (Hình 2.1).



Hình 2.1: Kiến trúc luồng thanh toán; đường đỏ là dữ liệu vượt trust boundary.

2.2 Bề mặt tấn công (attack surface)

Bảng 2.1: Các điểm nhận dữ liệu từ bên ngoài và rủi ro tương ứng.

Điểm vào	Dữ liệu	Hàm xử lý	Rủi ro
Webhook	Chữ ký HMAC-SHA256 (64 ký tự hex)	`parse_webhook_signature`	CWE-120
Webhook	Header token	`webhook_bypass_token`	CWE-798
API hỗ trợ viên	Mã PIN ghi đè	`support_override_pin`	CWE-798
Tạo đơn	Mã đơn hàng (20 ký tự)	`parse_order_id`	CWE-120
Webhook	Số tiền, phí vận chuyển	`calculate_total_payment`	CWE-190
API hoàn tiền	Số tiền hoàn, tổng tiền	`refund_ratio`	CWE-369
Nội bộ	Bản ghi audit, phiên, token thẻ	`write_audit_log`, `close_payment_session`, `release_card_token_buffer`	CWE-401/415/416

2.3 Mô hình mối đe dọa (STRIDE)

Bảng 2.2: Phân tích STRIDE cho luồng thanh toán.

Mối đe dọa	Kịch bản	Thuộc tính bị vi phạm	Yêu cầu
Spoofing	Gửi webhook giả "thanh toán thành công" bằng token bí mật	Authenticity	REQ-11
Tampering	Sửa số tiền trong payload làm tổng bị tràn số	Integrity	REQ-9
Repudiation	Mất bản ghi audit khi worker crash	Non-repudiation	NFR-3
Info. Disclosure	Tràn bộ đệm chữ ký làm lộ dữ liệu stack	Confidentiality	REQ-5
Denial of Service	Đơn treo mãi ở trạng thái timeout; hoàn tiền không kết thúc	Availability	REQ-2, REQ-13
Elevation of Priv.	PIN ghi đè hardcode của hỗ trợ viên	Authorization	REQ-11

2.4 Đặc tả yêu cầu

Đặc tả viết theo template Excel 10 sheet của học phần và được sinh tái lập bằng `scripts/build_payment_specs.py` thành hai file `payment_gateway_v0_spec.xlsx` và `payment_gateway_v1_spec.xlsx` trong thư mục `dataset/specs_`

`excel/`. Hai phiên bản dùng chung yêu cầu, thuộc tính và test case; chỉ khác sheet `MoHinhTrangThai` và cột `Loi` của `HamSinhMa`.

Bảng 2.3: Yêu cầu trong sheet `YeuCauChucNang` (rút gọn).

ID	Danh mục	Ưu tiên	Nội dung
REQ-1	safety	HIGH	Chỉ giao hàng khi đơn đã được xác nhận thu tiền
REQ-2	liveness	HIGH	Đơn timeout cuối cùng phải được đối soát xong
REQ-3	safety	HIGH	Hai worker không đồng thời giữ khóa sổ cái
REQ-4	liveness	HIGH	Không deadlock khi tranh chấp khóa hoặc khi hoàn tiền
REQ-5	memory	HIGH	`parse_webhook_signature` không tràn bộ đệm
REQ-6	memory	MEDIUM	`write_audit_log` giải phóng bộ nhớ mỗi bản ghi
REQ-7	memory	HIGH	`close_payment_session` không free hai lần
REQ-8	memory	HIGH	`release_card_token_buffer` không dùng sau khi free
REQ-9	safety	HIGH	Tổng tiền hàng + phí không tràn int32
REQ-10	functional	MEDIUM	`refund_ratio` không lỗi khi tổng tiền bằng 0
REQ-11	safety	HIGH	Không có mã bí mật hardcode vượt qua xác thực
REQ-12	safety	HIGH	Không ghi nhận thu tiền hai lần khi webhook bị gửi lại
REQ-13	liveness	MEDIUM	Mọi yêu cầu hoàn tiền cuối cùng phải hoàn tất
REQ-14	memory	MEDIUM	`parse_order_id` chỉ nhận mã đơn đúng độ dài

Bảng 2.4: Quy mô đặc tả (v0 / v1).

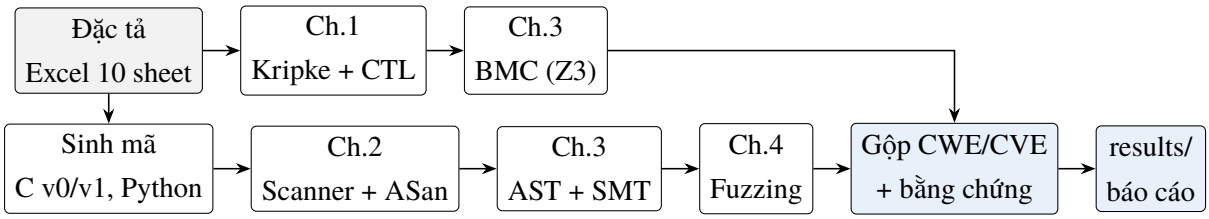
Thành phần		Số lượng
Yêu cầu chức năng / phi chức năng / ràng buộc		14 / 6 / 6
Trạng thái / chuyển tiếp của mô hình Kripke	14 / 23	(v1: 12 / 18)
Công thức CTL / cặp loại trừ lẫn nhau		8 / 5
Hàm sinh mã (C / Python)		6 / 3
Kiểm tra số học (Z3)		6
Test case truy vết		12
Số dòng mã C sinh ra (v0 + v1)		338

Đặc tả đáp ứng checklist của Hướng dẫn tạo spec Excel: đủ 10 sheet, mỗi sheet dạng bảng ≥ 5 dòng, phủ 4 danh mục yêu cầu, có cụm khóa `free/holds/conflict` với nhãn `violation`, công thức sống còn dạng $\mathbf{AG}(p \rightarrow \mathbf{AF}\, q)$, đủ 4 loại lỗi bộ nhớ C cùng hai loại hàm Python `divide` và `magic_value`; kích thước bộ đệm khớp các ràng buộc CON-1 đến CON-3.

3. Phương pháp thực hiện

Toàn bộ quy trình được tự động hóa trong một pipeline (Hình 3.1), chạy lại được bằng một lệnh với seed cố định; giao diện desktop gọi đúng pipeline này:

```
uv run python -m core.pipeline dataset/specs_excel/payment_gateway_v0_spec.
    xlsx \
   --compare dataset/specs_excel/payment_gateway_v1_spec.xlsx --seed 1 --
      budget 2000
```



Hình 3.1: Pipeline kiểm chứng end-to-end từ đặc tả tới báo cáo CWE.

3.1 Chương 1 — Mô hình Kripke và kiểm chứng CTL

Lý do chọn: REQ-1 đến REQ-4, REQ-12, REQ-13 là thuộc tính về *thứ tự trạng thái*, không kiểm tra được bằng một test case đơn lẻ mà cần xét *mọi* đường thực thi.

Cách áp dụng: mỗi dòng sheet `MoHinhTrangThai` là một trạng thái $s \in S$, cột `KhoiTao` xác định S_0, cột `Nhan` xác định $L(s) \subseteq AP$, cột `ChuyenDenCacTrangThai` sinh R. Mô hình gồm hai cụm độc lập: cụm khóa sổ cái (`ledger_free`, `webhook_holds`, `recon_holds`, `lock_conflict`) và vòng đời thanh toán. Module `core/ctl.py` parse công thức thành cây cú pháp (hỗ trợ lồng toán tử và $\neg, \wedge, \vee, \rightarrow$) rồi kiểm tra bằng thuật toán gán nhãn: **EX**, **EU**, **EG** tính bằng điểm bất động, các toán tử còn lại suy ra từ chúng. Công thức đúng khi mọi trạng thái khởi tạo thỏa nó; khi sai, công cụ sinh phản ví dụ — đường đi ngắn nhất tới trạng thái vi phạm với **AG**, hoặc đường lặp vô hạn (lasso) với **AF**. Công thức sai cú pháp bị báo *lỗi* chứ không bị coi là đạt.

Bảng 3.1: Thuộc tính CTL và truy vết tới yêu cầu.

#	Công thức	Loại	Yêu cầu
P1	**AG** $\neg$*violation*	Safety (mutex)	REQ-3
P2	*safe* $\rightarrow$ **EF** *webhook_locked*	Khả đạt	—
P3	**AG**(*shipped* $\rightarrow$ *paid*)	Safety	REQ-1
P4	**AG**(*timeout* $\rightarrow$ **AF**(*paid* $\vee$ *failed*))	Liveness	REQ-2
P5	**AG** $\neg$*double_capture*	Safety	REQ-12
P6	**AG** $\neg$(*processing* $\wedge$ *paid*)	Safety	—
P7	*created* $\rightarrow$ **EF** *shipped*	Khả đạt	—
P8	**AG**(*refunding* $\rightarrow$ **AF** *refunded*)	Liveness	REQ-13

P2 và P7 được viết dạng kéo theo vì mô hình có hai trạng thái khởi tạo thuộc hai cụm độc lập: **EF** *webhook_locked* viết trần sẽ sai tại `CREATED` dù cụm khóa hoàn toàn bình thường. Ngoài ra công cụ kiểm tra deadlock (trạng thái đạt được không có chuyển tiếp ra, CWE-833) và cả 5 cặp mệnh đề trong sheet `LoaiTruLanNhau`.

3.2 Chương 3 — Bounded Model Checking bằng Z3

Lý do chọn: duyệt tập trạng thái cho kết luận đúng/sai; BMC bổ sung đường phản ví dụ có *độ dài tối thiểu* và làm rõ khác biệt giữa "an toàn đến độ sâu k" và "an toàn".

Mã hóa: với mọi thuộc tính dạng **AG** φ (φ không chứa toán tử thời gian) và mọi cặp loại trừ lẫn nhau, $bad = S \setminus sat(\varphi)$ và

$$BMC_k \;=\; I(s_0) \;\wedge\; \bigwedge_{i=0}^{k-1} R(s_i, s_{i+1}) \;\wedge\; \bigvee_{i=0}^{k} bad(s_i) \tag{3.1}$$

trong đó mỗi s_i là một biến nguyên Z3 mang chỉ số trạng thái (`core/bmc.py`). Chạy $k = 0, 1, \ldots, CT$; nếu BMC_k SAT thì model của Z3 là phản ví dụ. Completeness threshold CT lấy bằng đường kính khả đạt (BFS từ S_0): mọi trạng thái đạt được đều đạt được trong $\leq CT$ bước, nên không vi phạm với mọi $k \leq CT$ nghĩa là thuộc tính đúng trên mọi độ dài. Mô hình v0 có $CT = 3$, mô hình v1 có $CT = 4$ (thêm trạng thái `REFUNDED`).

3.3 Chương 2 — Phân tích an toàn bộ nhớ trên mã C

Lý do chọn: các hàm C xử lý dữ liệu webhook và vòng đời phiên nằm trên attack surface (Bảng 2.1); lỗi bộ nhớ ở đây có mức rủi ro *Nghiêm trọng* theo ma trận rủi ro Chương 2.1.

Mỗi hàm được sinh hai bản cùng chữ ký: `*_v0.c` chứa lỗi khai báo, `*_v1.c` là bản vá. Hai lớp phân tích bổ sung cho nhau:

1. **Phân tích tĩnh** (`core/c_analyzer.py`): bỏ chú thích và chuỗi, tách từng hàm, duyệt theo thứ tự câu lệnh và theo dõi vòng đời từng con trỏ heap (cấp phát → kiểm tra NULL → giải phóng). Luật: hàm nguy hiểm (`strcpy`, `sprintf`, `gets`), chuỗi định dạng từ biến, `malloc` không kiểm tra NULL, kích thước cấp phát là tích, free hai lần, dùng sau free, thoát hàm hoặc gán lại con trỏ khi chưa free.
2. **Phân tích động** (`core/toolchain.py`): sinh harness `main()` cho từng hàm, biên dịch v0 và v1 bằng `clang -fsanitize=address,undefined -g -O0` và chạy với input độc hại: chuỗi dài gấp 8 lần bộ đệm (520 ký tự cho chữ ký), gọi hàm vòng đời bộ nhớ, và cặp `(INT_MAX, 1)` cho hàm cộng.

3.4 Chương 3 — Trích xuất AST và kiểm chứng SMT

AST (`core/static_ast.py`): với mỗi hàm và mỗi phiên bản, trích biến, điều kiện rẽ nhánh, vòng lặp, độ phức tạp McCabe $V(G) =$ số quyết định $+1$ và độ sâu lồng nhau; điều kiện rẽ nhánh của hàm Python được dùng lại cho concolic fuzzing.

SMT (`core/arithmetic_z3.py`): mỗi dòng sheet `KiemTraSoHoc` sinh bốn truy vấn Z3, với mọi biến ràng buộc trong miền int32:

1. tính thỏa được của điều kiện nghiệp vụ (ngưỡng chọn theo bảng int16 / uint16 / int32 của hướng dẫn);
2. giá trị đúng của biểu thức có thể vượt miền int32 hay không (lý thuyết số nguyên);
3. kết quả mà một `int` 32-bit của C thực sự tính ra — lý thuyết *bit-vector*, so sánh kết quả 32-bit với phép tính mở rộng 64-bit để lộ wrap-around;
4. thêm tiền điều kiện của bản vá (`b > 0 && a > INT_MAX - b...`) và chứng minh không còn input nào gây tràn (mong đợi UNSAT).

3.5 Chương 4 — Fuzzing

Ba hàm Python được fuzz trên cả v0 và v1 với cùng ngân sách 2 000 lần thử, seed $= 1$, input ban đầu $(1, 1)$ (vô hại, để lỗi phải được *khám phá*):

- `refund_ratio(refund_amount, total_amount)` — lỗi "nông" (chia cho 0);
- `webhook_bypass_token`, `support_override_pin` — nhánh vượt qua xác thực chỉ đạt được khi hai tham số bằng đúng cặp giá trị bí mật.

Ba chiến lược (`core/fuzzing.py`): **black-box** (số nguyên int32 ngẫu nhiên, 30% lấy từ tập giá trị biên), **coverage-guided** (đo line coverage bằng `sys.settrace`, đột biến bit flip / số học / giá trị biên / havoc / splice, chỉ giữ input mở thêm dòng mới — hill climbing), và **concolic** (chạy cụ thể đồng thời thu path condition dạng Z3 từ AST, phủ định lần lượt từng tiền tố để sinh input mới; phép chia được coi là nhánh ngầm *divisor* $\neq 0$). Oracle: ngoại lệ bất kỳ là crash (CWE-369 với chia cho 0); với hàm mã bí mật, trả về `True` là đã vượt qua xác thực (CWE-798). Mã sinh ra

chỉ được chạy sau khi kiểm tra AST chỉ chứa tập cú pháp an toàn.

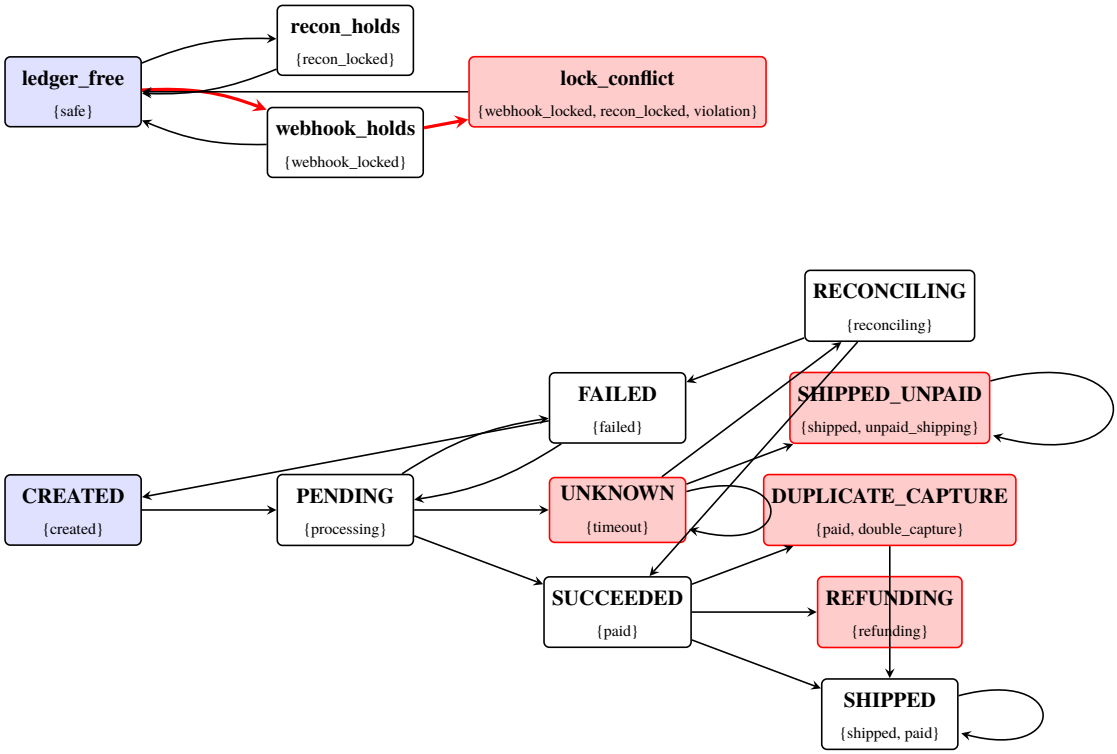
3.6 Gộp kết quả, ánh xạ CWE và đánh giá test case

Mọi phát hiện được chuẩn hóa thành bản ghi gồm: mã CWE (theo bảng ánh xạ của hướng dẫn: mutex → CWE-667, vi phạm CTL → CWE-693, deadlock → CWE-833, …), vị trí, công cụ, bằng chứng v0 và bằng chứng v1. Bản vá được coi là *đã xác minh* chỉ khi scanner v1 cho 0 finding, sanitizer v1 sạch và fuzzing v1 không tìm ra lỗi. Mỗi test case được đánh giá tự động PASS/FAIL từ câu "kết quả mong đợi" khi câu đó nêu tiêu chí kiểm tra được (một mã CWE tại một hàm, một công thức CTL kèm ĐÚNG/SAI, SAT/UNSAT, crash, …).

4. Kết quả và Phân tích

Toàn bộ log đầy đủ nằm trong `results/payment_gateway_v0/` và `results/payment_gateway_v1/` nộp kèm (Phụ lục B); chương này chỉ trích các dòng quan trọng nhất. Môi trường: macOS 26 (arm64), Python 3.11.15, z3-solver 4.16.0, Apple clang 17.0.0, seed $= 1$, ngân sách fuzzing 2 000. Toàn bộ pipeline chạy trong khoảng 6 giây, trong đó 5,5 giây là biên dịch và chạy sanitizer.

4.1 Kết quả Chương 1 — Kripke và CTL



Hình 4.1: Mô hình Kripke của đặc tả v0 (sinh tự động). Trạng thái đỏ là trạng thái vi phạm; cạnh đỏ là phản ví dụ đầu tiên (**AG** $\neg$*violation*).

Bảng 4.1: Kết quả kiểm chứng CTL trên đặc tả v0 và v1 (trích `comparison.md`).

Thuộc tính	v0	Phản ví dụ (v0)	v1
Deadlock	VI PHẠM	`REFUNDING` không có chuyển tiếp ra	ĐẠT
P1 **AG** $\neg$*violation*	VI PHẠM	ledger_free → webhook_holds → lock_conflict	ĐẠT
P2 *safe* → **EF** *webhook_locked*	ĐẠT	—	ĐẠT
P3 **AG**(*shipped* → *paid*)	VI PHẠM	CREATED → PENDING → UNKNOWN → SHIPPED_UNPAID	ĐẠT
P4 **AG**(*timeout* → **AF**(*paid* $\vee$ *failed*))	VI PHẠM	CREATED → PENDING → UNKNOWN → UNKNOWN (lặp)	ĐẠT
P5 **AG** $\neg$*double_capture*	VI PHẠM	CREATED → PENDING → SUCCEEDED → DUPLICATE_CAPTURE	ĐẠT
P6 **AG** $\neg$(*processing* $\wedge$ *paid*)	ĐẠT	—	ĐẠT
P7 *created* → **EF** *shipped*	ĐẠT	—	ĐẠT
P8 **AG**(*refunding* → **AF** *refunded*)	VI PHẠM	CREATED → PENDING → SUCCEEDED → REFUNDING (kẹt)	ĐẠT
Mutex `webhook_locked`/`recon_locked`	VI PHẠM	ledger_free → webhook_holds → lock_conflict	ĐẠT
4 cặp mutex còn lại	ĐẠT	—	ĐẠT

Phân tích.

- **P1 / mutex (CWE-667):** từ `webhook_holds` có cạnh sang `lock_conflict` — worker đối soát lấy khóa khi worker webhook chưa nhả. v1 xóa cạnh này.
- **P3 (CWE-693, lỗi nghiệp vụ CWE-840):** thiết kế v0 cho phép "giao hàng lạc quan" khi cổng timeout (`UNKNOWN` → `SHIPPED_UNPAID`): hàng đi khi tiền chưa về. v1 bắt buộc `UNKNOWN` → `RECONCILING`.
- **P4:** phản ví dụ là một *lasso*: `UNKNOWN` tự lặp (thăm dò cổng mãi) nên tồn tại một đường vô hạn không bao giờ đạt `paid` hay `failed`. Đây là lỗi mà không test case hữu hạn nào chứng minh được, nhưng model checking chỉ ra trực tiếp.
- **P5:** khi cổng gửi lại webhook (NFR-4), v0 ghi nhận thu tiền lần hai. v1 bỏ chuyển tiếp này (idempotency theo event ID).
- **P8 + deadlock (CWE-833):** `REFUNDING` không có chuyển tiếp ra. Nhờ quy ước "stutter" (trạng thái cụt được coi như tự lặp khi đánh giá CTL), P8 sai với phản ví dụ kẹt tại `REFUNDING`, đồng thời deadlock được báo riêng. Công cụ còn cảnh báo nhãn `refunded` không tồn tại trong mô hình v0 — dấu hiệu thiết kế thiếu trạng thái kết thúc.

4.1.1 Bounded Model Checking

Bảng 4.2: Kết quả BMC (trích `logs/bmc.txt`).

Thuộc tính	v0	k vi phạm	v1
AG $\neg$*violation*	UNSAFE	2	SAFE, đạt CT $= 4$
AG(*shipped* $\rightarrow$ *paid*)	UNSAFE	3	SAFE, đạt CT $= 4$
AG $\neg$*double_capture*	UNSAFE	3	SAFE, đạt CT $= 4$
AG $\neg$(*webhook_locked* $\wedge$ *recon_locked*)	UNSAFE	2	SAFE, đạt CT $= 4$
4 thuộc tính an toàn còn lại	SAFE, đạt CT $= 3$	—	SAFE, đạt CT $= 4$

```
BMC AG(shipped -> paid): UNSAFE
  k=0 BMC_k unsat
  k=1 BMC_k unsat
  k=2 BMC_k unsat
  k=3 BMC_k sat
  Counterexample:
    s_0 = CREATED
    s_1 = PENDING
    s_2 = UNKNOWN
    s_3 = SHIPPED_UNPAID
```

Đoạn mã 4.1: Truy vết BMC cho **AG**(*shipped* $\rightarrow$ *paid*) trên v0 (trích `logs/bmc.txt`).

BMC cho cùng phản ví dụ với model checking nhưng thêm thông tin *độ sâu tối thiểu*: không có đường vi phạm nào ngắn hơn 3 bước. Với v1, BMC_k UNSAT với mọi $k \leq 4 = CT$, nên kết luận an toàn là đầy đủ, không chỉ "an toàn đến độ sâu k".

4.2 Phân tích từng lỗ hổng và bản vá

Mỗi lỗ hổng được trình bày theo 5 mục: vị trí, nguyên nhân gốc rễ, bằng chứng tồn tại, cách vá, bằng chứng đã vá. Mã trích từ `results/payment_gateway_v0/source/`.

4.2.1 CWE-120 (Buffer Copy without Checking Size of Input) tại `parse_webhook_signature`

(1) Vị trí. `parse_webhook_signature_v0.c`, dòng 9; truy vết REQ-5, CON-1.

(2) Nguyên nhân gốc rễ. `strcpy` sao chép tới khi gặp `'\0'` mà không biết kích thước bộ đệm đích (65 byte trên stack). Chữ ký do bên ngoài kiểm soát (vượt trust boundary, Hình 2.1), nên chuỗi dài hơn 64 ký tự ghi đè vùng stack kề cận, có thể tới địa chỉ trả về (Chương 2.3).

(3) Bằng chứng lỗi tồn tại. Scanner: `dangerous-function` tại dòng 9. ASan với input 520 ký tự:

```
==46110==ERROR: AddressSanitizer: stack-buffer-overflow on address 0
    x00016bad2521
WRITE of size 521 at 0x00016bad2521 thread T0
   #1 in parse_webhook_signature parse_webhook_signature_v0.c:9
SUMMARY: AddressSanitizer: stack-buffer-overflow parse_webhook_signature_v0.c:9
```

Đoạn mã 4.2: Trích `logs/sanitizer_parse_webhook_signature_v0.log`.

(4) Cách vá.

```c
int parse_webhook_signature(
    const char *raw_signature) {
  char buffer[65];
  strcpy(buffer, raw_signature);
  printf("%s\n", buffer);
  return 0;
}
```

Đoạn mã 4.3: v0 — có lỗi

```c
int parse_webhook_signature(
    const char *raw_signature) {
  char buffer[65];
  if (raw_signature == NULL) return
      -1;
  size_t len = strnlen(raw_signature,
                  sizeof buffer);
  if (len >= sizeof buffer) return -1;
  memcpy(buffer, raw_signature, len);
  buffer[len] = '\0';
  printf("%s\n", buffer);
  return 0;
}
```

Đoạn mã 4.4: v1 — đã vá

Bản vá *từ chối* chữ ký quá dài thay vì cắt bớt âm thầm như `strncpy`: một chữ ký bị cắt có thể vẫn được so khớp một phần, còn từ chối loại bỏ hẳn nguyên nhân gốc. Cùng khuôn mẫu này được dùng cho `parse_order_id`, hàm không có lỗi khai báo: scanner và ASan đều cho 0 finding trên cả hai phiên bản (không báo nhầm).

(5) Bằng chứng đã vá. Scanner v1: 0 finding. ASan v1 với cùng input 520 ký tự: thoát mã 0, không có báo cáo, hàm trả về `rc=-1` (từ chối).

4.2.2 CWE-415 (Double Free) tại `close_payment_session`

(1) Vị trí. `close_payment_session_v0.c`, dòng 13; truy vết REQ-7.

(2) Nguyên nhân gốc rễ. Con trỏ `data` được `free` ở dòng 12 rồi lại ở dòng 13; lần thứ hai làm hỏng metadata của heap allocator và có thể bị khai thác để ghi tùy ý (Chương 2.2). Trong hệ thống thật, lỗi xuất hiện khi hai luồng cùng đóng một phiên.

(3) Bằng chứng lỗi tồn tại.

```
close_payment_session_v0.c:13: [HIGH] double-free CWE-415: free('data') lan
    hai (da free() tai dong 12)
==46141==ERROR: AddressSanitizer: attempting double-free on 0x6020000000d0 in
     thread T0:
   #1 in close_payment_session close_payment_session_v0.c:13
```

```
  freed by thread T0 here:
     #1 in close_payment_session close_payment_session_v0.c:12
  previously allocated by thread T0 here:
     #1 in close_payment_session close_payment_session_v0.c:8
```

Đoạn mã 4.5: Scanner và ASan trên v0.

Hai công cụ độc lập (tĩnh và động) chỉ ra cùng dòng 12/13.

(4) Cách vá.

```
1 data[0] = 'A';
2 free(data);
3 free(data);
4 return 0;
```

Đoạn mã 4.6: v0 — có lỗi

```
1 data[0] = 'A';
2 free(data);
3 data = NULL; /* free(NULL) vo hai */
4 return 0;
```

Đoạn mã 4.7: v1 — đã vá

(5) Bằng chứng đã vá. Scanner v1: 0 finding; ASan v1: sạch.

4.2.3 CWE-416 (Use After Free) và CWE-401 (Memory Leak)

`release_card_token_buffer_v0.c` dòng 12 ghi `data[0]` sau khi `free` ở dòng 11: scanner báo `use-after-free`, ASan báo `heap-use-after-free` tại cùng dòng. `write_audit_log_v0.c` dòng 12 thoát hàm khi vùng nhớ cấp phát ở dòng 8 chưa được giải phóng: scanner báo `memory-leak`. Bản v1 của cả hai hàm giải phóng đúng một lần rồi gán `NULL`; scanner và ASan v1 đều sạch. Lưu ý: CWE-401 chỉ có bằng chứng tĩnh vì LeakSanitizer không hỗ trợ macOS (xem Chương 5).

4.2.4 CWE-190 (Integer Overflow or Wraparound)

(1) Vị trí. `calculate_total_payment_v0.c` dòng 5 (`*out = amount + shipping_fee`) và 6 bộ đếm trong sheet `KiemTraSoHoc`; truy vết REQ-9, CON-4.

(2) Nguyên nhân gốc rễ. Cộng hai `int` có dấu vượt $2^{31} - 1$ là undefined behavior; thực tế thường wrap-around thành số âm, khiến tổng tiền cần thu âm hoặc rất nhỏ — lỗi số học dẫn thẳng tới lỗi nghiệp vụ (CWE-840).

(3) Bằng chứng. UBSan trên v0 với `(INT_MAX, 1)`:

```
calculate_total_payment_v0.c:5:19: runtime error: signed integer overflow:
   2147483647 + 1 cannot be represented in type 'int'
```

Đoạn mã 4.8: Trích `logs/sanitizer_calculate_total_payment_v0.log`.

Bảng 4.3: Kết quả SMT (trích `logs/z3.txt`); mọi biến trong miền int32.

Kiểm tra	Ngưỡng	Model tràn (thỏa điều kiện nghiệp vụ)	Wrap 32-bit	Sau vá
order_total_counter	int32	amount $=$ 2 147 483 647, shipping_fee $=1$	$-2\,147\,204\,848$	UNSAT
daily_transaction_counter	int16	tx_today $=$ 2 147 483 647, tx_backlog $=1$	$-2\,129\,640\,465$	UNSAT
active_session_counter	uint16	active $=$ 2 147 483 647, pending $=1$	$-2\,145\,843\,031$	UNSAT
refund_total_vnd	int32	refunded $=$ 2 147 483 647, pending $=1$	$-2\,143\,285\,231$	UNSAT
webhook_retry_counter	int16	today $=$ 2 147 483 647, backlog $=1$	$-2\,111\,565\,480$	UNSAT
settlement_batch_bytes	int32	batch $=$ 2 147 483 647, header $=1$	$-2\,145\,771\,516$	UNSAT

Cột "Wrap 32-bit" là kết quả mà một `int` của C thực sự tính ra (lý thuyết bit-vector). Ví dụ với `order_total_counter`, Z3 tìm được $1\,073\,758\,211 + 1\,074\,004\,237 = -2\,147\,204\,848$: hai khoản tiền dương cho tổng âm.

(4) Cách vá. Kiểm tra `(b > 0 && a > INT_MAX - b) || (b < 0 && a < INT_MIN - b)` trước khi cộng và trả lỗi (`calculate_total_payment_v1.c`).

(5) Bằng chứng đã vá. UBSan v1 sạch với cùng input. Với cả 6 bộ đếm, truy vấn tràn số kèm tiền điều kiện của bản vá trả về **UNSAT** — một *chứng minh toán học* rằng không tồn tại input int32 nào gây tràn, mạnh hơn mọi test case.

4.2.5 CWE-369 (Divide By Zero) tại `refund_ratio` và CWE-798 (Hard-coded Credentials)

Bằng chứng của hai lớp lỗi này đến từ fuzzing (Mục 4.3). `refund_ratio` v0 chia `refund_amount // total_amount` không kiểm tra mẫu số; v1 trả 0 khi mẫu số bằng 0. `webhook_bypass_token` và `support_override_pin` v0 trả `True` khi hai tham số bằng cặp hằng số (7788, 9999) và (4821, 7306) — cửa hậu hardcode. v1 so khớp với bí mật cấu hình ngoài mã nguồn bằng `hmac.compare_digest` (so sánh thời gian hằng) và từ chối khi chưa cấu hình.

4.3 So sánh các chiến lược fuzzing

Bảng 4.4: Fuzzing với ngân sách 2 000 lần thử, seed $= 1$ (trích `logs/fuzz.csv`).

Hàm (v0)	Chiến lược	Lần thử chạy	Số lần lỗi	Lần đầu	Input gây lỗi
`refund_ratio`	Black-box	2000	47	48	(2147483646, 0)
`refund_ratio`	Coverage-guided	2000	24	6	(1, 0)
`refund_ratio`	Concolic	3	1	2	$(-2147483648, 0)$
`webhook_bypass_token`	Black-box	2000	0	—	—
`webhook_bypass_token`	Coverage-guided	2000	0	—	—
`webhook_bypass_token`	Concolic	5	1	4	(7788, 9999)
`support_override_pin`	Black-box	2000	0	—	—
`support_override_pin`	Coverage-guided	2000	0	—	—
`support_override_pin`	Concolic	5	1	4	(4821, 7306)
Cả 3 hàm, bản v1	Cả 3 chiến lược	2000	0	—	—

Phân tích.

- `refund_ratio` là lỗi "nông": mọi chiến lược đều tìm ra. Black-box tìm được nhờ 30% input lấy từ tập giá trị biên có chứa 0 — không có tập này, xác suất trúng `total_amount = 0` ngẫu nhiên chỉ là 2^{-32} mỗi lần.
- Với mã bí mật, xác suất một cặp int32 ngẫu nhiên trúng đúng cặp là 2^{-64}: black-box thất bại hoàn toàn sau 2 000 lần, đúng như Chương 4.5.
- Coverage-guided *cũng* thất bại: điều kiện `header_token == 7788` không mở dòng mã mới cho tới khi khớp chính xác, nên tín hiệu coverage không tăng dần — đây là "vấn đề magic byte" (Chương 4.3–4.6) mà AFL giải bằng CmpLog/laf-intel.
- Concolic thu path condition $\mathit{header_token} = 7788 \wedge \mathit{secret_token} = 9999$ từ AST rồi giải trực tiếp bằng Z3, tìm ra cửa hậu ở lần chạy thứ 4 và dừng khi hết nhánh để phủ định (chỉ 5 lần chạy).
- Trên v1, concolic dừng sau 1 lần chạy vì hàm gọi `os.environ`/`hmac` mà engine không mô hình hóa được — giới hạn "environment modeling" của symbolic execution (Chương 4.6), được ghi nhận trong `logs/fuzz_events.txt`.

4.4 Bảng tổng hợp trước/sau

Bảng 4.5: Tổng hợp lỗ hổng và trạng thái trước/sau (rút gọn từ `before_after.md` và `comparison.md`).

CWE	Vị trí	Trạng thái v0	Trạng thái v1	Công cụ xác minh
CWE-120	`parse_webhook_signature`	ASan stack-buffer-overflow dòng 9 (WRITE 521 byte)	Từ chối input > 64 ký tự; 0 finding, ASan sạch	Scanner + ASan
CWE-401	`write_audit_log`	Scanner: không free trước return, dòng 12	0 finding	Scanner
CWE-415	`close_payment_session`	ASan attempting double-free dòng 13	0 finding, ASan sạch	Scanner + ASan
CWE-416	`release_card_token_buffer`	ASan heap-use-after-free dòng 12	0 finding, ASan sạch	Scanner + ASan
CWE-190	`calculate_total_payment`	UBSan signed overflow dòng 5	UBSan sạch	UBSan
CWE-190	6 bộ đếm	Z3 SAT, wrap thành số âm	Z3 UNSAT với tiền điều kiện	Z3
CWE-369	`refund_ratio`	ZeroDivisionError, 47/2000 lần (black-box)	0/2000 cả 3 chiến lược	Fuzzing
CWE-798	`webhook_bypass_token`	Bypass với (7788, 9999), concolic lần 4	0/2000	Concolic
CWE-798	`support_override_pin`	Bypass với (4821, 7306), concolic lần 4	0/2000	Concolic
CWE-667	Khóa sổ cái	Mutex vi phạm, phản ví dụ độ dài 2	ĐẠT; BMC SAFE đạt CT	CTL + BMC
CWE-693	P3, P4, P5, P8	4 thuộc tính CTL sai, có phản ví dụ	Cả 4 ĐẠT	CTL + BMC
CWE-833	`REFUNDING`	Deadlock	Không deadlock	Kripke

4.5 Truy vết test case

Bảng 4.6: Test case truy vết từ đặc tả, đánh giá tự động (trích `comparison.md`).

ID	Yêu cầu	Kết quả mong đợi	v0	v1
TC-1	REQ-3, REQ-4	AG(!violation) ĐÚNG và không có trạng thái deadlock	FAIL	PASS
TC-2	REQ-1	AG(shipped -> paid) ĐÚNG	FAIL	PASS
TC-3	REQ-2	AG(timeout -> AF(paid | failed)) ĐÚNG	FAIL	PASS
TC-4	REQ-12, NFR-4	AG(!double_capture) ĐÚNG	FAIL	PASS
TC-5	REQ-13	AG(refunding -> AF(refunded)) ĐÚNG và không có deadlock	FAIL	PASS
TC-6	REQ-5, CON-1	Không phát hiện CWE-120 tại parse_webhook_signature	FAIL	PASS
TC-7	REQ-6, 7, 8	Không phát hiện CWE-401, CWE-415, CWE-416	FAIL	PASS
TC-8	REQ-9, CON-4	Không phát hiện CWE-190 tại calculate_total_payment	FAIL	PASS
TC-9	REQ-9	Z3 SAT khi chưa kiểm tra biên và UNSAT sau khi thêm tiền điều kiện	PASS	PASS
TC-10	REQ-10	Không crash (ZeroDivisionError) tại refund_ratio	FAIL	PASS
TC-11	REQ-11, CON-6	Không phát hiện CWE-798	FAIL	PASS
TC-12	REQ-14, CON-2	Không phát hiện CWE-120 tại parse_order_id	PASS	PASS
Tổng			2/12	12/12

TC-12 PASS trên cả hai phiên bản là kiểm tra *âm tính*: hàm không có lỗi khai báo thì công cụ không được báo nhầm. TC-9 PASS trên v0 vì nó kiểm tra chính khả năng chứng minh của Z3, không kiểm tra mã v0.

5. Thảo luận và Hạn chế

5.1 Tính đúng đắn (sound) và đầy đủ (complete) của từng kỹ thuật

Theo định nghĩa của học phần: *sound* — không có false negative khi kết luận an toàn; *complete* — không có false positive.

Bảng 5.1: Tính chất của các kỹ thuật đã dùng trong đề tài.

Kỹ thuật	Sound?	Complete?	Ghi chú trong đề tài
CTL trên Kripke	Có	Có	Chỉ đúng với *mô hình*, không phải mã thật
BMC (chưa đạt CT)	Có trong k	Không	
BMC (đạt CT)	Có	Có	v0: CT $= 3$, v1: CT $= 4$, đều đã đạt
Static scanner	Không	Không	Heuristic theo mẫu, trong một hàm, không phân tích alias
ASan / UBSan	Không	Có trên đường đã chạy	Chỉ một input độc hại cho mỗi hàm
Z3 (int32, bit-vector)	Có	Có	Với biểu thức đã mô hình hóa
Black-box fuzzing	Không	Có (lỗi tìm được luôn thật)	
Coverage-guided fuzzing	Không	Có	Kẹt ở điều kiện magic value
Concolic fuzzing	Không	Có	Chỉ tập con cú pháp; không mô hình hóa được thư viện ngoài

5.2 Hạn chế

- **Khoảng cách mô hình – cài đặt.** CTL/BMC chứng minh tính chất của *mô hình trạng thái* do chúng tôi viết tay từ yêu cầu; nếu mô hình không phản ánh đúng hệ thống thanh toán thật thì kết luận "v1 an toàn" không tự động đúng cho hệ thống đó.
- **Mã sinh từ đặc tả.** Các hàm C/Python được sinh theo khuôn mẫu nên nhỏ và đơn giản (6–10 dòng mỗi hàm). Việc công cụ phát hiện được lỗi cài có chủ đích không chứng minh nó sẽ phát hiện được lỗi tinh vi hơn trong mã viết tay (vd lỗi qua alias hay qua nhiều hàm).
- **Static scanner** tự viết, dựa trên mẫu: không sound, không complete. Vì vậy mọi finding bộ nhớ đều được đối chiếu với ASan — trừ CWE-401.
- **Môi trường.** Trên macOS Apple Silicon, LeakSanitizer không được hỗ trợ nên CWE-401 chỉ có bằng chứng tĩnh; chạy pipeline trên Linux sẽ bổ sung log leak. cppcheck không được cài

trong môi trường chạy, nên bước này được bỏ qua.

- **Sanitizer** chỉ chạy một input độc hại cho mỗi hàm; nó xác nhận lỗi tồn tại chứ không chứng minh v1 an toàn với mọi input — phần đó do Z3 (số học) và phân tích bản vá đảm nhận.
- **Fuzzing** không chứng minh sự vắng mặt của lỗi. Black-box tìm được lỗi chia cho 0 phần lớn nhờ tập giá trị biên; concolic dừng khi gặp lời gọi thư viện ngoài.
- **Mô hình đơn luồng.** Hai worker được mô hình bằng cụm trạng thái khóa viết tay, chưa dùng tổ hợp song song (parallel composition) để sinh tự động mọi cách đan xen.

5.3 Phản biện và bài học

Nếu bắt đầu lại, sẽ thiết kế khác điều gì? Đưa idempotency (event ID của webhook), giới hạn thời gian chờ ở trạng thái timeout và trạng thái kết thúc của hoàn tiền vào đặc tả *ngay từ đầu*. Ba lỗi thiết kế P4, P5, P8 đều là "thiếu một trạng thái/chuyển tiếp" — những lỗi rẻ nhất để sửa ở giai đoạn đặc tả (shift-left) và đắt nhất nếu phát hiện khi vận hành.

Lỗ hổng nào nghi ngờ còn tồn tại nhưng chưa xác nhận? Xử lý webhook đồng thời trên nhiều đơn (race giữa hai lần gửi lại cùng một sự kiện) chưa được mô hình hóa bằng tổ hợp song song; rò rỉ bộ nhớ trên nhánh lỗi của mã viết tay (không phải mã sinh ra) chưa được kiểm tra động do thiếu LeakSanitizer trên macOS.

Bài học lớn nhất về phương pháp luận. Không một kỹ thuật đơn lẻ nào đủ: CTL tìm lỗi thiết kế mà không test hữu hạn nào chứng minh được (lasso của P4), Z3 chứng minh bản vá số học cho *mọi* input, còn fuzzing và sanitizer xác nhận lỗi trên mã chạy thật. Bài học thứ hai đến từ chính quá trình làm: phiên bản đầu của bộ kiểm tra CTL âm thầm báo "đạt" với mọi công thức nó không hiểu (ví dụ $\mathbf{AG}\,\neg(a \wedge b)$) — một công cụ như vậy *không sound*. Chúng tôi viết lại bộ kiểm tra, buộc công thức lỗi cú pháp phải bị báo lỗi, và kiểm tra nó bằng 30 mô hình ngẫu nhiên đối chiếu với phép duyệt vét cạn.

6. Kết luận

Đề tài đã áp dụng tuần tự kỹ thuật của cả bốn chương lên một hệ thống cổng thanh toán, với mọi kết luận đi kèm bằng chứng công cụ:

- **Mô hình hóa (Ch.1):** trên thiết kế v0, 7 trong 14 kiểm tra (8 công thức CTL, 5 cặp mutex, deadlock) bị vi phạm, mỗi vi phạm có phản ví dụ — gồm giao hàng khi chưa thu tiền, thu tiền hai lần, chờ vô hạn khi timeout và hoàn tiền không kết thúc. Thiết kế v1 đạt cả 14.
- **BMC (Ch.3):** 4 thuộc tính an toàn UNSAFE trên v0 với phản ví dụ độ dài tối thiểu 2–3; trên v1 mọi thuộc tính SAFE và đã đạt completeness threshold (CT $= 4$).
- **Bộ nhớ (Ch.2):** 4 lỗi bộ nhớ C được scanner phát hiện, 3 trong số đó được ASan xác nhận tại đúng dòng; bản v1 cho 0 finding và sanitizer sạch.
- **SMT (Ch.3):** cả 6 bộ đếm có thể tràn int32 (wrap thành số âm); tiền điều kiện của bản vá được chứng minh loại bỏ hoàn toàn tràn số (UNSAT).
- **Fuzzing (Ch.4):** concolic tìm ra cả hai cửa hậu sau 4 lần chạy, trong khi black-box và coverage-guided không tìm ra sau 2 000 lần; bản v1 cho 0 lỗi trên cả 3 chiến lược.
- **Tổng hợp:** 21 lỗ hổng có bằng chứng trên v0 (0 lỗi chỉ khai báo), test case 2/12 PASS; trên v1 không còn lỗ hổng chưa vá, test case 12/12 PASS.

Hướng phát triển: mô hình hóa xử lý webhook đồng thời bằng tổ hợp song song, và áp dụng CBMC / LeakSanitizer trên Linux cho mã thanh toán viết tay thay vì mã sinh từ đặc tả.

Tài liệu tham khảo

[1] Armin Biere, Alessandro Cimatti, Edmund Clarke, and Yunshan Zhu. Symbolic model checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pages 193–207, 1999.

[2] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pages 337–340, 2008.

[3] Patrice Godefroid, Michael Y. Levin, and David Molnar. Automated whitebox fuzz testing. In *Network and Distributed System Security Symposium (NDSS)*, 2008.

[4] Dorothy Graham, Erik van Veenendaal, Isabel Evans, and Rex Black. *Foundations of Software Testing: ISTQB Certification*. Cengage Learning, 2020.

[5] MITRE. Common weakness enumeration (cwe). `https://cwe.mitre.org`. Truy cập TODO.

[6] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. AddressSanitizer: A fast address sanity checker. In *USENIX Annual Technical Conference*, pages 309–318, 2012.

[7] Quang Dũng Vũ, Thanh Bình Trịnh, and Trung Sơn Đoàn. Bài giảng cse703093 – an toàn phần mềm, chương 1–4, 2026. Khoa Hệ thống thông tin, Trường Đại học Phenikaa.

A. Hướng dẫn chạy lại

```
cd source
uv sync
uv run pytest # 103 test
uv run python scripts/build_payment_specs.py # sinh lai dac ta v0/v1 (tuy
    chon)
uv run python -m core.pipeline dataset/specs_excel/payment_gateway_v0_spec.
    xlsx \
   --compare dataset/specs_excel/payment_gateway_v1_spec.xlsx --seed 1 --
      budget 2000
uv run python main.py # giao dien desktop
```

Môi trường đã dùng (`results/payment_gateway_v0/environment.txt`): macOS 26 (Darwin 25.2.0, arm64), Python 3.11.15, z3-solver 4.16.0, Apple clang 17.0.0. Không cần cài thêm công cụ nào ngoài Python và một trình biên dịch C; thiếu trình biên dịch thì bước sanitizer được bỏ qua và ghi rõ.

B. Danh mục bằng chứng nộp kèm

File (trong `results/payment_gateway_v0/`)	Nội dung
`report.md`	Toàn bộ kết quả theo chương, phân tích từng CWE theo 5 mục
`before_after.md`	Bảng trước/sau đầy đủ 21 dòng
`comparison.md`	So sánh thiết kế đặc tả v0 $\rightarrow$ v1
`summary.json`	Toàn bộ số liệu dạng máy đọc được
`logs/ctl.txt`, `logs/bmc.txt`	Kết quả CTL (kèm phản ví dụ) và BMC theo từng k
`logs/scan.txt`, `logs/scan.json`	Finding của static scanner trên v0 và v1
`logs/sanitizer_*_v0.log`, `_v1.log`	Lệnh biên dịch, input và log ASan/UBSan đầy đủ
`logs/z3.txt`	Truy vấn và model Z3 cho 6 kiểm tra số học
`logs/ast.csv`	Biến, điều kiện, McCabe, độ sâu lồng của 9 hàm $\times$ 2 phiên bản
`logs/fuzz.csv`, `logs/fuzz_events.txt`	Kết quả và sự kiện của 18 chiến dịch fuzzing
`logs/tests.csv`	Đánh giá 12 test case
`source/`	Mã sinh ra `*_v0` và `*_v1`
`figures/kripke.png`, `kripke.tex`	Sơ đồ Kripke có tô phản ví dụ

C. Khai báo sử dụng công cụ AI

Theo quy định liêm chính học thuật của học phần, các phần sau có sự hỗ trợ của công cụ AI (Claude — Anthropic):

- **[TODO: Mô tả phần mã nguồn có hỗ trợ AI và mức độ (gợi ý / viết nháp / sinh toàn bộ), ví dụ các module core/ctl.py, core/bmc.py, core/fuzzing.py, core/c_analyzer.py, giao diện gui/.]**
- **[TODO: Mô tả phần báo cáo có hỗ trợ AI (ví dụ: khung LaTeX, dàn ý, bản nháp các chương).]**

Mọi kết quả thực nghiệm trong báo cáo được sinh ra bằng cách chạy công cụ (thư mục `results/`) và đã được sinh viên kiểm tra lại.